



Education

Lab 4.3. CI/CD Using Argo Workflows

This lab will give you an overview of implementing a CI/CD pipeline using Argo Workflow. Imagine you have a microservices-based application deployed on a Kubernetes cluster. You want to implement a CI/CD pipeline using Argo Workflows to automate the build, test, and deployment process for each microservice.

Objective

- Define workflows that include various steps such as building, testing, and deploying applications.
- Incorporate testing steps into your workflows to ensure the quality of your applications.
- Run unit tests, integration tests, and other relevant tests as part of the CI/CD process.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
 - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set up a cluster
- Argo Workflows CLI (optional).

Create a CI/CD Pipeline with Argo Workflow

1. Create Argo Workflow

We can start by creating and managing our workflows as code. We will create a Python application workflow with the following steps:

- Build: The build step builds the image with the latest changes, using a Python 3 image for this scenario.
- Tests: The test step mounts a volume with test files and runs unit tests with the Python unittest library.

- Deployment: The deploy step runs the Python container and prints deploy. Normally, this step would involve pushing the tested code to a container registry, like AWS ECR or Harbor, and then deploying it to the production environment.

This lab provides a basic setup for Argo Workflow CI/CD. Depending on your specific requirements, you may want to enhance the workflow with additional steps, such as testing, linting, or pushing the Docker image to a registry.

Define the Argo Workflow manifest (`workflow-ci.yaml`) that describes the sequence of tasks and dependencies.

```
cat <<EOF > workflow-ci.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  name: python-app
spec:
  entrypoint: python-app
  templates:
  - name: python-app
    steps:
    - - name: build
      template: build
    - - name: test
      template: test
    - - name: deploy
      template: deploy
  - name: build
    container:
      image: python:3.11
      command: [python]
      args: ["-c", "print('build')"]
  - name: test
    container:
      image: python:3.11
      command: [python]
      args: ["-m", "unittest", "discover", "-s", "/app/tests"]
      volumeMounts:
      - name: test-volume
        mountPath: /app/tests
    volumes:
    - name: test-volume
      hostPath:
        path: /path/to/tests
```

```
- name: deploy
  container:
    image: python:3.11
    command: [python]
    args: ["-c", "print('deploy')"]
EOF
```

2. Deploy the Workflow

Deploy the workflow by running the following command:

```
$ kubectl -n argo apply -f workflow-ci.yaml
```

This will create an Argo Workflow with the name **python-app** along with their respective templates and steps.

3. Inspect the Workflow

Now, let's execute the **argo get** command to retrieve information about the workflow, including its status:

```
$ argo -n argo get python-app
```

Output:

```
Name:                python-app
Namespace:           argo-workflows
ServiceAccount:      unset (will run with the default ServiceAccount)
Status:              Succeeded
Conditions:
  PodRunning         False
  Completed          True
Created:             Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Started:             Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Finished:            Mon Jan 08 00:08:39 +0100 (20 minutes ago)
Duration:            48 seconds
Progress:            3/3
ResourcesDuration:   26s*(1 cpu),16s*(100Mi memory)
```

STEP	TEMPLATE	PODNAME	DURATION
MESSAGE			
✓ python-app	python-app		
—✓ build	build	python-app-build-3831382643	19s

```
|—✓ test      test      python-app-test-461837862    4s
|—✓ deploy    deploy     python-app-deploy-988129288  5s
```

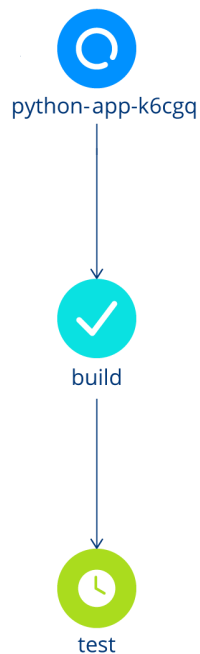
The work can be observed in the cluster as a set of pods (one for each step). Run the following command:

```
$ kubectl get pods -n argo
```

Output:

NAME	READY	STATUS	RESTARTS
argo-server-66c9577ddc-cf9zf	1/1	Running	0
python-app-build-3831382643	0/2	Completed	0
python-app-deploy-988129288	0/2	Completed	0
python-app-test-461837862	0/2	Completed	0
workflow-controller-5bb95ffc45-pvp9d	1/1	Running	0

You can also see the Workflow progress in Argo UI:



CI/CD Workflow progress overview

We can also view the logs:

```
$ argo -n argo logs python-app
```

Output:

```
python-app-build-3831382643: build

python-app-build-3831382643: time="2025-11-03T21:05:42.251Z"
level=info msg="sub-process exited" argo=true error="<nil>"

python-app-test-461837862:

python-app-test-461837862:
-----

python-app-test-461837862: Ran 0 tests in 0.000s

python-app-test-461837862:

python-app-test-461837862: OK

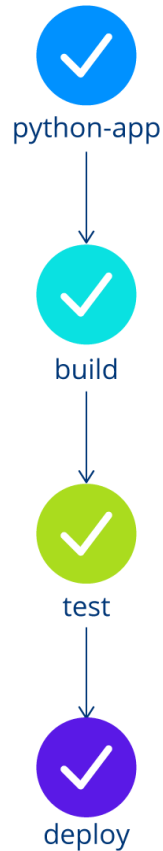
python-app-test-461837862: time="2025-11-03T21:05:54.210Z" level=info
msg="sub-process exited" argo=true error="<nil>"

python-app-deploy-988129288: deploy

python-app-deploy-988129288: time="2025-11-03T21:06:04.315Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

Refresh the UI of Argo Workflows. You should see a new workflow that is processing.

Refresh again after a few minutes and you should see the following screen with a completed workflow.



CI/CD Workflow overview

4. Summary

In this chapter, we completed a lab setup by deploying a CI/CD pipeline using Argo Workflow to show how to establish a robust and automated pipeline for building, testing, and deploying applications in Kubernetes, promoting a DevOps culture and enhancing software delivery practices. Here we mention a few benefits of such an implementation:

- **Automation and efficiency:** Argo Workflows automates the CI/CD pipeline, reducing manual intervention and enhancing efficiency.
- **Consistency and reproducibility:** With Argo Workflows, CI/CD processes are defined declaratively, ensuring consistency across multiple runs and environments.
- **Scalability:** Argo Workflows provides scalability, allowing CI/CD processes to scale with the growing needs of the application.
- **Visibility and monitoring:** Argo Workflows' UI and CLI provide visibility into workflow execution, allowing teams to monitor progress and troubleshoot issues.
- **Flexibility and customization:** Argo Workflows offers flexibility in defining custom workflows, allowing teams to tailor CI/CD pipelines to their specific requirements.